DOKUZ EYLÜL UNIVERSITY

Engineering Faculty — Department of Computer Engineering

CME 2204 Algorithm Analysis

2025-2026 Spring

Assignment 1: Comparison of Tim Sort and Quick Sort

Student Name: Baran Öztürk

Student Number: 2024510100

Supervisor: Asst. Prof. Feriştah Dalkılıç

İzmir, 20 March 2026

Performance Comparison of Tim Sort and Quick Sort on Large Integer Datasets

Baran Öztürk
Dept. of Computer Eng.
Dokuz Eylül University
baran.o@ogr.deu.edu.tr
2024510100

Abstract

This paper presents an empirical performance comparison between Tim Sort and Quick Sort applied to one million integers across four distinct input distributions: random, semi-ordered, increasing, and decreasing. The Tim Sort implementation employs insertion sort for sub-arrays of size 64 or fewer, followed by merge sort for larger partitions. Five pivot selection strategies are evaluated for Quick Sort: first element, last element, middle element, random element, and median-of-three. Execution times are measured in milliseconds, and sorted outputs are persisted to disk in plain text (.txt) format. Results demonstrate that Tim Sort consistently outperforms Quick Sort on partially sorted data, while certain pivot strategies lead to severe degradation—particularly the first and last element strategies on already-sorted inputs. The median-of-three strategy provides the most robust Quick Sort performance across all input distributions.

Keywords—Tim Sort; Quick Sort; pivot strategy; sorting algorithms; algorithm analysis; empirical performance evaluation; input distribution; Java

I. INTRODUCTION

Sorting is one of the most fundamental operations in computer science, underpinning the efficiency of search, database, and data-processing systems. Although theoretically O(n log n) algorithms are optimal for comparison-based sorting, real-world performance varies dramatically depending on the algorithm's design and the structure of the input data.

This assignment investigates two prominent sorting algorithms—Tim Sort and Quick Sort—by measuring their wall-clock execution times on 1,000,000 integers drawn from four input distributions. Tim Sort is the default sorting algorithm in Python, Java (Arrays.sort), and Android due to its ability to exploit pre-existing order. Quick Sort remains one of the fastest in-place sorters in practice due to cache efficiency, but its performance depends critically on pivot selection.

The remainder of this report is organized as follows: Section II describes the algorithms, Section III details the implementation, Section IV presents the experimental setup and hardware specifications, Section V reports results, Section VI discusses findings, and Section VII concludes the paper.

II. ALGORITHMS

A. TIM SORT

Tim Sort [1] is a hybrid stable sorting algorithm derived from Merge Sort and Insertion Sort. It was designed by Tim Peters in 2002 and is optimized for real-world data, which frequently contains naturally ordered subsequences ('runs').

The algorithm proceeds as follows:

1. Divide the array into runs of a fixed size (minrun). A minrun of 32 or 64 is typical; in this implementation minrun = 64.
2. Sort each run using Insertion Sort. If the total array length does not exceed minrun, only Insertion Sort is applied.
3. Merge adjacent sorted runs using Merge Sort. After each merge pass, the merged subarray size doubles.

Tim Sort achieves O(n log n) worst-case and O(n) best-case (already sorted) time complexity, with O(n) auxiliary space.

B. QUICK SORT

Quick Sort [2] is a divide-and-conquer algorithm that selects a pivot element, partitions the array into elements less than and greater than the pivot, and recursively sorts the sub-arrays. Its average-case complexity is O(n log n), but it degrades to $O(n^2)$ when the pivot selection produces highly unbalanced partitions—notably on sorted or reverse-sorted input with the first or last element as pivot.

Five pivot selection strategies are evaluated in this work:

- First element – always selects arr[low] as pivot.
- Last element – always selects arr[high] as pivot.
- Middle element – selects arr[low + (high−low)/2].
- Random element – selects a uniformly random index in [low, high].
- Median-of-three – selects the median of arr[low], arr[mid], and arr[high].

III. IMPLEMENTATION

The complete source code is publicly available on GitHub [3]. The project is implemented in Java and consists of three source files: SortingAlgorithms.java, FileOperations.java, and Main.java. All timing measurements capture only the sort operation itself, excluding file I/O and array initialisation.

A. SORTINGALGORITHMS.JAVA

This class encapsulates both sorting algorithms. The quickSort method accepts a PivotStrategy enum, which provides a type-safe and extensible way to define pivot selection logic and it delegates pivot selection to the pivot helper, which swaps the chosen element to arr[high] before calling the standard Lomuto-scheme partition function. The mergeSort method implements Tim Sort by switching to insertionSort when the sub-array length falls at or below the threshold x (= 64).

Listing 1: Quick Sort Core Logic

```
public enum PivotStrategy { FIRST, LAST, MIDDLE, RANDOM, MEDIAN }

private static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) { i++; swap(arr, i, j); }
    }
    swap(arr, i + 1, high);
    return i + 1;
}

public static void quickSort(int[] arr, int low, int high, PivotStrategy ps) {
    if (low < high) {
        pivot(arr, low, high, ps);
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1, ps);
        quickSort(arr, pi + 1, high, ps);
    }
}
```

Listing 2: Tim Sort (mergeSort + insertionSort)

```
public static void mergeSort(int[] arr, int low, int high, int x) {
    if (x < high - low + 1) {
        int mid = low + (high - low) / 2;
```

```
            mergeSort(arr, low, mid, x);
            mergeSort(arr, mid + 1, high, x);
            merge(arr, low, mid, high);
        } else {
            insertionSort(arr, low, high);  // run size threshold = 64
        }
    }

    private static void insertionSort(int[] arr, int low, int high) {
        for (int i = low + 1; i <= high; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= low && arr[j] > key) { arr[j+1] = arr[j]; j--; }
            arr[j+1] = key;
        }
    }
```

B. MAIN.JAVA

The main class loads random.txt and semi_ordered.txt via FileOperations.loadFileIntoArray, constructs increasing and decreasing arrays programmatically, clones each array before every sorting call to ensure identical inputs, measures execution time with System.currentTimeMillis(), and writes sorted results to 24 output files following the naming convention algorithm_datatype_out.txt.

Listing 3: Timing Wrapper Methods in Main.java

```
    public static void quickSort_Operations(int[] arr,
            SortingAlgorithms.PivotStrategy ps, String fileName) {
        int[] temp = arr.clone();
        long start = System.currentTimeMillis();
        SortingAlgorithms.quickSort(temp, 0, arr.length-1, ps);
        long end = System.currentTimeMillis();
        System.out.println("Time: " + (end - start) + " ms");
        FileOperations.writeArrayToFile(fileName, temp);
    }

    public static void timSort_Operations(int[] arr, int x, String fileName) {
        int[] temp = arr.clone();
        long start = System.currentTimeMillis();
        SortingAlgorithms.mergeSort(temp, 0, arr.length-1, x);
        long end = System.currentTimeMillis();
        System.out.println("Time: " + (end - start) + " ms");
        FileOperations.writeArrayToFile(fileName, temp);
    }
```

C. FILEOPERATIONS.JAVA

This utility class provides two static methods. loadFileIntoArray reads one integer per line into a pre-allocated array of 1,000,000 elements using a BufferedReader. writeArrayToFile serializes a sorted array to disk in plain text (.txt) format, writing each element on a separate line via a BufferedWriter. Both methods handle IOException gracefully.

IV. EXPERIMENTAL SETUP

A. HARDWARE SPECIFICATIONS

Processor: AMD Ryzen 7 7840HS w/ Radeon 780M Graphics (3.80 GHz)
RAM: 32,0 GB (31,3 GB usable) 5600 MT/s

Storage: 954 GB SSD
Operating System: Windows 11 Pro 64-bit
JDK Version: OpenJDK 25.0.2

B. INPUT DATASETS

Four integer arrays of size 1,000,000 are used:

1. Random – loaded from random.txt; uniformly distributed integers.
2. Semi-ordered – loaded from semi_ordered.txt; partially sorted sequences.
3. Increasing – generated in code as arr[i] = i.
4. Decreasing – generated in code as arr[n−i−1] = i.

Each array is cloned before every sorting call so that all algorithms operate on an identical, unmodified input. Tim Sort uses a run size of 64 for efficient performance.

V. RESULTS

Table 1 presents the measured execution times (in milliseconds) for each algorithm–input combination. The file I/O time is excluded from all measurements, as instructed.

Table 1. Performance Comparison Matrix (ms)

Algorithm	RANDOM INTEGERS	SEMI-ORDERED INTEGERS	INCREASING INTEGERS	DECREASING INTEGERS
Tim Sort	192 ms	94 ms	46 ms	60 ms
Quick Sort – First element	131 ms	112 ms	366672 ms	522964 ms
Last element	119 ms	94 ms	866110 ms	606434 ms
Middle element	127 ms	100 ms	26 ms	29 ms
Random element	141 ms	111 ms	62 ms	68 ms
Median element	127 ms	94 ms	26 ms	57 ms

VI. DISCUSSION

A. TIM SORT

Tim Sort is expected to deliver the fastest results on already-sorted (increasing) and semi-ordered inputs because the insertion-sort phase exploits existing order within each 64-element run, resulting in very few comparisons and movements before the merge phase. On random data, Tim Sort's overhead from the two-phase hybrid structure keeps it competitive but may not surpass well-tuned Quick Sort variants. On decreasing-order data, insertion sort performs its worst case on each run ($O(x^2)$ comparisons per run), potentially increasing total time, although the merge phase remains O(n log n).

B. QUICK SORT – PIVOT STRATEGIES

The first-element and last-element pivot strategies are expected to exhibit $O(n^2)$ behavior on increasing and decreasing inputs, respectively, because each partition step produces sub-arrays of sizes 0 and n−1, maximally unbalanced. This leads to a recursion depth of O(n), which may also trigger a StackOverflowError for n = 1,000,000. To prevent StackOverflowError during worst-case scenarios (O(n) recursion depth) with the first-element pivot strategy, the JVM stack size was increased

using the **-Xss200m** flag. The middle-element strategy performs well on sorted inputs because the middle pivot evenly divides already-sorted data, achieving O(n log n) behaviour. However, adversarial inputs can still cause degradation.
The random-element strategy avoids worst-case behaviour in expectation, since no fixed input distribution can reliably produce bad pivots. Its average complexity remains O(n log n) across all tested distributions.

The median-of-three strategy is theoretically superior to a single-element selection because it reduces the probability of an extreme pivot. It provides consistent performance across all four input distributions and is widely considered the pragmatic best choice for Quick Sort in production code.

C. BEST AND WORST CASES SUMMARY

The empirical results confirm theoretical predictions with striking clarity. Among all 24 algorithm–input combinations, the fastest execution was achieved jointly by Quick Sort with the middle-element pivot and Quick Sort with the median-of-three pivot, both completing in 26 ms on the increasing-order dataset. Tim Sort recorded the second-best result on the same input at 46 ms, benefiting from its adaptive insertion-sort phase on pre-sorted runs.

The worst-case results were observed for Quick Sort with the last-element pivot on the increasing-order dataset (866,110 ms ≈ 14.4 minutes) and Quick Sort with the first-element pivot on the decreasing-order dataset (522,964 ms ≈ 8.7 minutes). Both cases correspond to the $O(n^2)$ degenerate behavior described in Section VI-B, where every partition step produces maximally unbalanced sub-arrays. The recursion depth of O(n) for n = 1,000,000 required the JVM stack to be enlarged with the –Xss200m flag to avoid a StackOverflowError.

On random data, all Quick Sort variants outperformed Tim Sort (192 ms), with the last-element pivot being the fastest at 119 ms. This is consistent with the well-known cache-efficiency advantage of Quick Sort on uniformly distributed inputs, where the probability of a severely unbalanced partition is low. On semi-ordered data, Tim Sort (94 ms) matched the last-element and median-of-three Quick Sort variants, confirming its strength on inputs with pre-existing partial structure.

In summary, the median-of-three strategy is the best overall Quick Sort variant: it achieves the joint-fastest time on increasing data (26 ms), ties for best on semi-ordered data (94 ms), and never degenerates on any tested distribution. Tim Sort is the best choice when the input ordering is unknown or partially structured, offering predictable performance in the range of 46–192 ms across all four distributions. The first-element and last-element pivot strategies should be avoided for any input that may be sorted or reverse-sorted, as their runtimes become impractical at n = 1,000,000.

VII. CONCLUSION

This study compared Tim Sort and five variants of Quick Sort on 1,000,000-element integer arrays with four distinct ordering patterns. The results confirm well-established theoretical expectations: Tim Sort excels on partially sorted data due to its adaptive nature, while Quick Sort performance is highly sensitive to pivot selection. Among Quick Sort strategies, the median-of-three approach consistently provides the most robust performance, avoiding $O(n^2)$ degenerate cases that afflict first- and last-element pivots on sorted inputs. The random pivot strategy also performs reliably in practice, though with a minor overhead from the random number generation calls.

Future work could explore Introsort—a hybrid of Quick Sort and Heap Sort that falls back to Heap Sort when recursion depth exceeds a threshold—as well as Parallel Tim Sort and other cache-oblivious algorithms.

REFERENCES

[1] T. Peters, "timsort.txt," CPython source tree, 2002. [Online]. Available: https://github.com/python/cpython/blob/main/Objects/listsort.txt
[2] C. A. R. Hoare, "Algorithm 64: Quicksort," Communications of the ACM, vol. 4, no. 7, p. 321, 1961.
[3] B. Öztürk, "ComparisonOfTimSortAndQuickSort," GitHub, Mar. 2026. [Online]. Available: https://github.com/Ztrk06/ComparisonOfTimSortAndQuickSort